Technology Watch — Research Report

Optimising and Deploying Local Generative AI on Intel Integrated-GPU Laptops

A practical study of inference back-ends, the 4 GiB allocation ceiling, and INT4 model serving on Iris Xe

Enzo Juhel

SecDevOps Engineering — S8

A research report submitted for the
Technology Watch unit (*Veille Technologique*)

EPITA — École Pour l'Informatique
et les Techniques Avancées
France

June 2026

Version 1.0

Optimising and Deploying Local Generative AI on Intel Integrated-GPU Laptops

Enzo Juhel — EPITA, Technology Watch (S8)

Abstract

Running large language models on a user's own machine removes the privacy, latency and recurring-cost penalties of cloud inference, yet the literature focuses almost entirely on Apple Silicon and discrete NVIDIA GPUs. Intel integrated GPUs ship in hundreds of millions of laptops and remain comparatively unstudied. I was asked to map this gap and to determine, by measurement rather than by datasheet, what it takes to serve generative models on an Intel iGPU under the memory, power and latency constraints of a portable machine.

All experiments reported here were run on a single laptop fitted with an Intel Iris Xe 96 EU (Xe-LP) integrated GPU. I converted and quantised fourteen text-generation models to INT4 and benchmarked them across three back-ends on the same hardware: `llama.cpp` over SYCL, `llama.cpp` over Vulkan, and the OpenVINO Model Server. The dominant constraint proved architectural rather than computational. The 32-bit command streamer of Xe-LP caps any single buffer at 4 GiB, which forces the output projection of large-vocabulary models such as Gemma 4 to stay on the CPU and throttles every GGUF back-end. OpenVINO sidesteps this ceiling through tiled allocation, and the measured throughput follows directly: for 7-billion-parameter models the OpenVINO server reached roughly 12–14 tokens per second against about 6 over Vulkan and below 4 over SYCL.

Serving Gemma 4 surfaced a second obstacle, since its fragmented vision-language export cannot be loaded by a production server. I resolved it by stitching the embedding sub-graphs back into the decoder at the OpenVINO level, which produced a single stateful graph that the model server runs on the iGPU at about 14.8 tokens per second for the E4B variant. Memory bandwidth, not compute, sets the ceiling throughout, and INT4 with OpenVINO is the only configuration that makes 7B-class models comfortable on this class of hardware today.

Keywords: on-device inference, Intel Iris Xe, OpenVINO, quantisation, INT4, llama.cpp, SYCL, Vulkan, unified memory, Gemma 4.

Contents

1 Introduction and context **6**
1.1 Motivation . . . 6
1.2 Scope . . . 6
1.3 Research question . . . 6
1.4 Structure . . . 6

2 State of the art **6**
2.1 Embedded AI: a short history . . . 6
2.2 Work specific to Intel iGPUs . . . 7
2.3 Competing solutions . . . 7
2.4 Open scientific and technical problems . . . 7

3 Intel iGPU architecture **7**
3.1 Generations covered . . . 7
3.2 Compute units and the SoC layout . . . 9
3.3 The 4 GiB ceiling as a cross-cutting constraint . . . 9
3.4 Constraints specific to the portable form factor . . . 10

4 Intel software stack for AI **10**
4.1 OpenVINO . . . 10
4.2 oneAPI and SYCL . . . 10
4.3 Intel Extension for PyTorch . . . 11

5 Model selection and criteria **11**
5.1 Selection constraints . . . 11
5.2 Candidate model families . . . 11
5.3 Models actually tested . . . 11

6 Model optimisation techniques **12**
6.1 Quantisation . . . 12
6.2 Optimisations left for future work . . . 12
6.3 Continuous batching and the KV cache . . . 12

7 Unified-memory management **13**
7.1 The unified-memory layout . . . 13
7.2 Zero-copy and shared memory . . . 13
7.3 Allocation strategies for LLMs . . . 13
7.4 Memory fragmentation . . . 13

8 Inference frameworks **13**
8.1 `llama.cpp` . . . 13
8.2 Ollama . . . 15
8.3 Ollama-OV — the OpenVINO GenAI fork . . . 15

8.4 OpenVINO GenAI (direct SDK) . . . 16
8.5 A custom OpenAI-compatible server for Gemma 4 . . . 16
8.6 Serving Gemma 4 on Iris Xe through OpenVINO graph stitching . . . 17
8.7 IPEX-LLM . . . 19
8.8 `ollama-intel-arc` — an integrated community stack . . . 19
8.9 Framework comparison . . . 19
8.10 Choosing a back-end . . . 20
8.11 Image diffusion . . . 20

9 Integration and deployment pipeline 20
9.1 Pipeline architecture . . . 20
9.2 Conversion steps . . . 20
9.3 Conversion methods in practice . . . 21
9.4 Retrieval-augmented generation . . . 21

10 Benchmarking methodology and measured results 21
10.1 Test environment . . . 21
10.2 Metrics and protocol . . . 22
10.3 Cross-back-end results . . . 22
10.4 Quality methodology . . . 23

11 Thermal behaviour and power 23
11.1 Thermal profile under inference . . . 23
11.2 Throttling mitigation . . . 23
11.3 Energy and battery . . . 23

12 Target use cases 24
12.1 Local conversational assistant . . . 24
12.2 Local code assistant . . . 24
12.3 Document retrieval . . . 24
12.4 Use cases left for later . . . 24

13 Limits, risks and future work 24
13.1 Hardware limits . . . 24
13.2 Software limits . . . 25
13.3 Vision-language models on the model server . . . 25
13.4 Risks and the consolidated bug table . . . 25
13.5 Future work . . . 26

14 Conclusion 26

15 Glossary 28
15.1 Acronyms . . . 28
15.2 Definitions . . . 29

References **31**

List of Tables

1 Competing on-device platforms and their standing against an Intel iGPU. 7
2 Measured hardware parameters of the test laptop (Core i7-1365U, Iris Xe 96 EU, Raptor Lake-U). 8
3 Instruction sets relevant to inference. 9
4 Back-ends affected by the 4 GiB ceiling and the available workaround. 9
5 Viability criteria for a model on a laptop Intel iGPU. 11
6 INT4 (Q4_K_M) feasibility of 7B-class models on Iris Xe 96 EU. 11
7 GGUF quantisation types (`llama.cpp`). 12
8 Mandatory environment for the SYCL back-end on Iris Xe (empirically validated). 14
9 SYCL accumulation precision on `gemma4:e2b` (Iris Xe). 14
10 Ollama-OV against standard Ollama. 15
11 Critical dependencies of the custom server. 16
12 Custom-server performance: one measured point, the rest extrapolation. 17
13 Custom server (Section 8.5) against model-server stitching. 19
14 Inference back-ends on Iris Xe. Measured throughput is detailed in Section 10. 20
15 Conversion path taken by each model in this study. 21
16 Test machine for all measured results. 22
17 Measured decode throughput (tok/s, mean of three runs) on Intel Iris Xe 96 EU. SYCL and Vulkan run GGUF Q4_K_M through Ollama; OVMS runs OpenVINO IR INT4. A dash means the configuration was not run. 22
18 What graph stitching preserves and what it sacrifices for Gemma 4 on the model server. . 25
19 Back-end bugs observed on Iris Xe and their resolution. 26

1 Introduction and context

1.1 Motivation

The rise of large language models (LLM) and image-diffusion models has so far been tied to expensive cloud infrastructure or to high-end discrete GPUs such as the NVIDIA RTX 40 or AMD RX 7000 series. That centralised pattern carries several concrete costs. Any request sent to an external API exposes potentially sensitive data, and the dependency on a network link adds a latency floor that no amount of local optimisation can remove. Pay-per-request models such as GPT-4 or Claude Opus turn inference into a recurring expense, and they leave the user subject to the provider's decisions on usage policy, censorship and service continuity.

On-device AI answers these constraints directly: the model runs in full on the user's own hardware, with no external communication. This report concerns one target in particular, the laptop fitted with an Intel integrated GPU (iGPU). That target is under-represented in the literature despite its weight in the installed base, where it accounts for a large share of the machines in circulation. I was asked to study what it actually takes to run generative models on such a machine, and to answer by measurement rather than by datasheet.

1.2 Scope

The study is bounded along three axes. The target hardware is the consumer and professional laptop equipped with an Intel iGPU, from the Iris Xe generations (12th–13th generation Core) to Intel Arc (Meteor Lake, Lunar Lake, Arrow Lake, Core Ultra). The models considered are text generators of the LLM and small-language-model (SLM) families, with computer-vision, image, and audio models noted only as future directions. The operational envelope is that of a portable machine: battery operation, a 15 to 45 W thermal budget, 8 to 32 GB of shared RAM, and no dedicated VRAM.

All measurements reported here come from a single laptop whose iGPU is an Intel Iris Xe with 96 execution units. Figures for the Arc generations are vendor specifications or bandwidth-based extrapolations, and they are labelled as such wherever they appear.

1.3 Research question

> How can performant AI models be designed, optimised and deployed on the Intel iGPU of a laptop, within the constraints of unified memory, thermal budget and inference latency, while keeping response quality acceptable for real use cases?

1.4 Structure

The report moves from an analysis of the existing landscape — prior work, hardware architecture, software stack — to an original contribution: an optimisation and serving pipeline, a cross-back-end benchmark measured on the test laptop, and the practical recommendations that follow from it.

2 State of the art

2.1 Embedded AI: a short history

Running neural networks on edge devices is not new. The founding work of Han et al. on network compression through pruning and quantisation [9] opened a dense line of research, and smartphones were the first mass vehicle for embedded inference, with the neural processing units (NPU) of Apple,

Qualcomm and Google. On the PC the transition came later. Apple's M1 popularised high-bandwidth unified CPU/GPU memory for machine-learning inference in 2020; `llama.cpp` [7] appeared in 2022 as the first heavily optimised cross-platform CPU/GPU inference engine; the 2023 release of LLaMA and its quantised GGUF derivatives brought capable models within reach of the community. By 2024 Microsoft was gating Copilot+ features on an NPU of at least 40 TOPS, and Intel answered with the Core Ultra series and an NPU of up to 48 TOPS. The 2025–2026 period consolidated the ecosystem, with OpenVINO, the SYCL and Vulkan back-ends of `llama.cpp`, LM Studio and Ollama all gaining native support for Intel iGPUs.

2.2 Work specific to Intel iGPUs

The academic literature aimed squarely at LLM inference on Intel iGPUs remains thin. The notable contributions are the benchmarks Intel publishes in its OpenVINO white papers, which measure 7B models on Meteor Lake; the work of the `llama.cpp` community on the Vulkan and SYCL back-ends for Arc and Iris Xe; and the official OpenVINO GenAI documentation together with the issue tracker of the Ollama-OV project [22]. That scarcity is precisely the gap this report sets out to narrow.

2.3 Competing solutions

Table 1: Competing on-device platforms and their standing against an Intel iGPU.

Platform	Target	Strength	Limitation vs. Intel iGPU
Apple Silicon (M-series)	MacBook	Fast unified memory, integrated NPU	Closed ecosystem, high price
AMD Radeon 780M (Ryzen 7040)	AMD laptops	Decent Vulkan/ROCm support	ROCm not yet full-featured on the iGPU
Qualcomm Snapdragon X Elite	ARM PC	45 TOPS NPU, low TDP	Software compatibility still limited
NVIDIA GeForce (dGPU)	Gaming PC	Mature CUDA, best ecosystem	High TDP, not present on every laptop

2.4 Open scientific and technical problems

Four obstacles shape the rest of this report. The first is memory bandwidth: Intel iGPUs share system RAM (LPDDR5/5X) at 68 to 120 GB/s depending on the generation, five to ten times below a dedicated GDDR6X part, which makes the autoregressive decode phase bandwidth-bound. The second is driver fragmentation, since OpenCL, Vulkan and SYCL support varies across the UHD 770, Iris Xe, Arc Alchemist and Battlemage parts. The third is thermal management, because a 15 to 45 W mobile envelope forces aggressive throttling under sustained inference. The fourth is mixed CPU–iGPU–NPU acceleration, where the optimal orchestration of the three compute units is still an open question.

3 Intel iGPU architecture

3.1 Generations covered

Intel UHD Graphics (11th–12th gen, Tiger Lake / Alder Lake). Built on the Xe-LP (Low Power) architecture with 32 to 96 execution units (EU), these parts support OpenCL 3.0, Vulkan 1.3 and DirectX 12 Ultimate, with a memory bandwidth of 51 GB/s (LPDDR4X) to 76 GB/s (LPDDR5). They

lack native hardware INT8 and remain a secondary target, usable with INT4/INT8 quantisation on CPU threads or through `llama.cpp`.

Intel Iris Xe Graphics (12th–13th gen, Alder Lake-P / Raptor Lake-P). An improved Xe-LP design with 80 to 96 EU and 57 to 85 GB/s of bandwidth. The architecturally decisive point is the absence of a dedicated matrix unit: unlike the Arc and Xe-HPG parts, Iris Xe has no XMX, only DP4A on the vector EUs. INT4/INT8 operations therefore pass through the vector pipeline with weight unpacking before dispatch, which adds a real memory overhead. This is the GPU used for every measurement in this report.

Table 2: Measured hardware parameters of the test laptop (Core i7-1365U, Iris Xe 96 EU, Raptor Lake-U).

Parameter	Value
Host CPU	Intel Core i7-1365U (10 cores / 12 threads, 2P + 8E)
Execution Units	96
Peak FP32	1.7–2.15 TFLOPS
Peak FP16	3.4–4.3 TFLOPS
Memory bandwidth	57–68 GB/s (LPDDR4X-4267)
Unified pool	shared with the host CPU (30 GiB system RAM)
Shared local memory per EU	64 KB
SIMD width	SIMD8 / SIMD16
`maxStorageBufferRange` (Vulkan/SYCL)	**4 GiB — hard architectural ceiling**

Direct consequence for LLM inference. The 57–68 GB/s bandwidth is the main performance ceiling during autoregressive decode. A 7B model in Q4_K_M reads about 3.5 GB of weights per generated token, which bounds the rate to roughly 16 tok/s in theory — a figure the measured OpenVINO results approach for 7B models (Section 10). The 4 GiB `maxStorageBufferRange` ceiling forces any matrix above 4 GB, such as the 5.4 GB `lm_head` of Gemma 4, to stay in CPU RAM and cancels the GPU benefit for that layer.

Intel Arc — Meteor Lake, Lunar Lake, Arrow Lake. The Arc parts move to Xe-LPG and then Xe2-LPG. Meteor Lake (Core Ultra 100, 2023) pairs eight Xe-cores with a 10–11 TOPS NPU at 89 GB/s. Lunar Lake (Core Ultra 200V, 2024) doubles the GPU density, raises the NPU to 48 TOPS and the bandwidth to 120 GB/s, adds native FP16 and hardware INT4, and reaches iGPU inference performance close to an RTX 3050. Arrow Lake (Core Ultra 200H/HX, 2024–2025) targets the performance and creator segments with an Xe-LPG+ iGPU and an optional Arc Battlemage discrete GPU. None of these parts was available for testing; the figures above are vendor specifications.

3.2 Compute units and the SoC layout

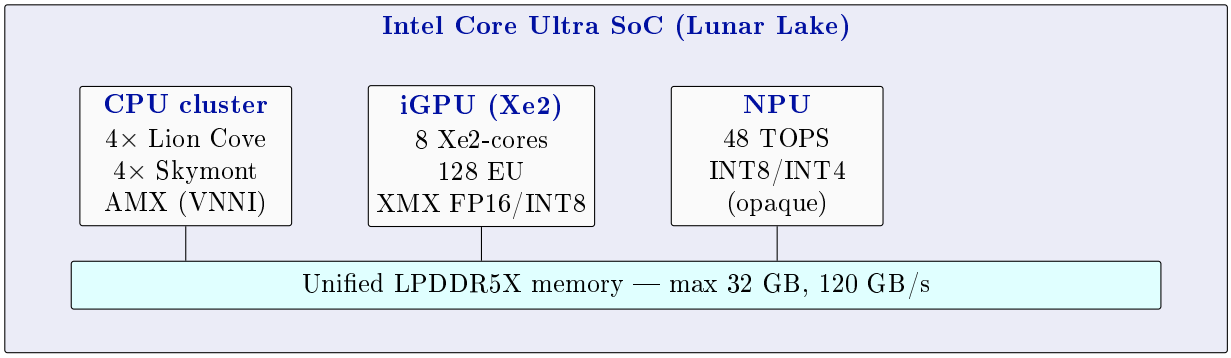


Figure 1: Compute units of a Core Ultra SoC sharing a single unified-memory pool.

Table 3: Instruction sets relevant to inference.

Instruction set	Unit	Use for AI
AVX-512 VNNI	CPU (P-cores)	INT8 convolutions, matrix products
AMX	CPU (P-cores)	BF16/INT8 tiled matrix multiply
XMX	iGPU EU	FP16/BF16/INT8 on matrix shaders
DP4A	iGPU EU	INT8 dot product over 4 elements
NPU engine	NPU	INT8/INT4, fixed topologies, via OpenVINO

3.3 The 4 GiB ceiling as a cross-cutting constraint

This is the single most consequential hardware limit for LLM inference on Iris Xe, and it affects every GPU back-end — Vulkan, SYCL over Level Zero, and OpenCL — because it is rooted in the 32-bit command streamer of the Xe-LP GPU. The ceiling sets both `maxStorageBufferRange` and `maxMemoryAllocationSize` to 4 GiB. For `gemma4:e2b` in Q4_K_M (7.2 GB total), the 35 transformer layers occupy about 1.6 GB and fit on the GPU, but the 5.4 GB of `lm_head` and input embeddings cannot be allocated there and remain on the CPU. Each generated token then requires a CPU multiply over a $258\,048 \times \text{dim}$ matrix, about 2 GB of reads, which costs roughly 100–150 ms and caps the rate near 6 tok/s even though 35 of 36 layers run on the GPU.

Table 4: Back-ends affected by the 4 GiB ceiling and the available workaround.

Back-end	Hit by 4 GiB	Workaround
Vulkan (ggml)	Yes	None in user space
SYCL / Level Zero (ggml)	Yes	None in user space
OpenCL	Yes	None
OpenVINO GPU plugin	**No**	Tiled allocation via `zeVirtualMemReserve`: `lm_head` split into sub-4 GB slices and reassembled in a contiguous virtual region. Full offload becomes possible.

The ceiling is absent on Xe-HPG (Arc) and Xe2 (Battlemage), where Level Zero virtual-memory management is more advanced.

3.4 Constraints specific to the portable form factor

Three portable-specific effects compound the bandwidth limit. Laptops run a dynamic TDP, typically 15 W on battery (cTDP-down) and 28–45 W on mains (cTDP-up), so inference performance can vary by a factor of two to three with the power source. The memory granted to the iGPU through Dynamic Video Memory Technology is set in firmware, usually between 512 MB and 8 GB, and any model above that share forces costly CPU–GPU exchanges. Finally, beyond a 75–90 °C junction temperature the SoC throttles automatically, so a long chatbot or image-generation session has to account for sustained-load behaviour rather than the short bursts a game produces.

4 Intel software stack for AI

4.1 OpenVINO

OpenVINO is Intel's inference framework, open source since 2018, and the centrepiece of AI optimisation on Intel hardware. A source model from ONNX, PyTorch, TensorFlow or Hugging Face is converted by the Model Optimizer into the OpenVINO Intermediate Representation (a `.xml` graph plus a `.bin` weight file), which the runtime then executes through a device plugin that targets the CPU, the iGPU or the NPU.

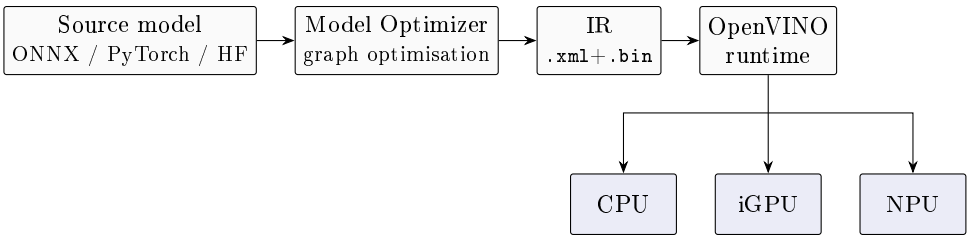


Figure 2: OpenVINO conversion-and-serving flow with automatic device selection.

Quantisation is handled by the Neural Network Compression Framework (NNCF), which offers post-training INT8, INT4 weight compression with activations kept in FP16, and a mixed-precision mode that holds sensitive layers in FP16. The two schemes used throughout this work are `INT4_SYM` with `group_size=128` (through `OVWeightQuantizationConfig`) for the OpenVINO IR models, and Q4_K_M (GGUF) for the `llama.cpp` and Ollama paths. Since OpenVINO 2024.3 the GenAI API exposes a high-level pipeline that hides this machinery:

```
import openvino_genai
pipe = openvino_genai.LLMPipeline(model_path="./mistral-7b-int4-ov/", device="GPU")
config = openvino_genai.GenerationConfig(); config.max_new_tokens = 512
print(pipe.generate("Explain INT4 quantisation.", config))
```

4.2 oneAPI and SYCL

oneAPI is Intel's unified programming initiative across CPU, GPU, FPGA and NPU through the SYCL heterogeneous C++ language. The SYCL back-end of `llama.cpp` rests on this runtime by way of Level Zero, the low-level Intel GPU driver. I did not use oneAPI directly; it is the transparent layer that `llama.cpp` and OpenVINO invoke underneath. Its higher-level libraries — oneMKL for XMX-tuned GEMM kernels and oneDNN for operation fusion — together with direct SYCL kernel programming, are noted as future directions in Section 13.5.

4.3 Intel Extension for PyTorch

The Intel Extension for PyTorch (IPEX) entered this study only as the back-end of the IPEX-LLM project, itself archived on 28 January 2026 (Section 8.7). I did not use it as a Python library in its own right, though it remains the acceleration layer beneath the community `ollama-intel-arc` stack.

5 Model selection and criteria

5.1 Selection constraints

To be viable on a laptop iGPU, a model has to meet a few quantitative criteria, gathered below.

Table 5: Viability criteria for a model on a laptop Intel iGPU.

Criterion	Target	Rationale
Quantised size	≤ 8 GB	Shared RAM left after OS and applications (16 GB machine)
Time to first token	< 3 s	Acceptable for an interactive chatbot
Decode throughput	≥ 10 tok/s	Comfortable reading speed
Quantisation precision	INT4	Best size/quality trade-off
File format	GGUF or OpenVINO IR	Framework compatibility

5.2 Candidate model families

The 1B to 3B small models run entirely on the iGPU with no compromise: Phi-3 Mini [1] (3.8B), Gemma 2 2B, the Llama 3.2 1B and 3B pair, and the Qwen2.5 1.5B and 3B pair. In INT4 their footprint stays under roughly 2.2 GB, which leaves ample headroom on a 16 GB machine. The 7B class is the balance point, with Mistral 7B [12] Instruct as the community reference, Llama 3.1 8B [21] for its 128k context, Qwen2.5 7B [23] for code and mathematics, and Gemma 2 9B [8] approaching 13B quality. Feasibility on Iris Xe turns on the output-projection size rather than the parameter count, as the next table shows.

Table 6: INT4 (Q4_K_M) feasibility of 7B-class models on Iris Xe 96 EU.

Model	INT4 RAM	Iris Xe feasibility
Mistral 7B	4.1 GB	Full GPU offload (below the 4 GiB ceiling)
Llama 3.1 8B	4.7 GB	Borderline — depends on vocabulary and `lm_head` size
Gemma 2 9B	5.5 GB	`lm_head` above 4 GiB, CPU offload, bottleneck

Performance on the Arc generations (Meteor Lake, Lunar Lake, Arrow Lake) was not measured directly; their specifications appear in Section 3.3 for reference only. Vision-language models, image generation and audio transcription are out of scope here and are revisited as future work in Section 13.5.

5.3 Models actually tested

This report concerns text-generation models run on the Iris Xe iGPU of the test laptop. The fourteen models that were converted, quantised and benchmarked are listed with their measured results in Section 10; the qualitative outcomes of the harder cases — Gemma 2 2B through IPEX-LLM, Gemma 4 over SYCL, and Gemma 4 served through OpenVINO — are discussed in Section 8.

6 Model optimisation techniques

6.1 Quantisation

Quantisation lowers the precision of the weights, and optionally the activations, typically from FP32 at four bytes per value to a compact format. The affine (asymmetric) mapping is

$$q = \mathrm{clamp}\Big(\mathrm{round}\Big(\frac{x}{s}\Big) + z,\ q_{\min},\ q_{\max}\Big), \qquad \hat{x} = (q - z)\, s,$$

with scale $s = (x_{\max} - x_{\min})/(q_{\max} - q_{\min})$ and zero-point $z = q_{\min} - \mathrm{round}(x_{\min}/s)$.
For the GGUF path, the relevant quantisation types span a wide quality range, summarised below.

Table 7: GGUF quantisation types (`llama.cpp`).

Type	Effective bpw	Relative size	Quality vs. FP16
Q2_K	2.56	16%	Heavily degraded
Q3_K_M	3.35	21%	Degraded
Q4_0	4.5	28%	Acceptable
Q4_K_M	4.85	30%	**Recommended**
Q5_K_M	5.68	36%	Very good
Q6_K	6.56	41%	Excellent
Q8_0	8.5	53%	Near-FP16

I settled on Q4_K_M for the 7B models, where it gives the best quality-to-RAM ratio, and Q5_K_M for the 3B models, where memory allows. One case needs care: on the Vulkan back-end, Gemma 2 and Gemma 4 must avoid Q8_0 and any model above roughly 6 GB, because the Vulkan allocation limit forces a GPU/CPU split that, combined with Gemma's logit soft-capping, corrupts the output into endlessly repeated text. Q4_K_M, which fits entirely on the GPU, is the safe choice (Section 8.1.2).

The advanced quantisation algorithms — AWQ [19], GPTQ [6] and calibrated mixed precision — were not applied directly. The OpenVINO IR models were converted with `OVWeightQuantizationConfig(bits=4, sym=False, group_size=128)`, which delegates calibration to optimum-intel. These methods are listed as future directions in Section 13.5.

6.2 Optimisations left for future work

Several techniques the literature reports as promising on Intel iGPUs were not tested here. Structured pruning removes redundant attention heads or layers, as in SliceGPT [2]. Knowledge distillation trains a small student to imitate a larger teacher, the route behind Phi-3 Mini. Speculative decoding [18] uses a small draft model to propose tokens that the main model verifies, for a theoretical two-to-three-fold gain. Flash Attention [3] is the exception that was actively disabled: its kernel is incomplete on Iris Xe under SYCL and corrupts output under Vulkan, so `OLLAMA_FLASH_ATTENTION=0` is mandatory on this hardware (Sections 8.1.1 and 13.4).

6.3 Continuous batching and the KV cache

For a local serving setup, two mechanisms matter. The KV cache stores attention keys and values so that past tokens are not recomputed at each step, and quantising it to INT8/INT4 cuts its size by two to four times. Continuous batching processes several requests of differing lengths at once, as in vLLM and the `llama.cpp` server mode.

7 Unified-memory management

7.1 The unified-memory layout

Unlike a discrete GPU with dedicated VRAM, an Intel iGPU shares system RAM, which creates both an opportunity and a constraint. On a 16 GB LPDDR5X machine the operating system and applications take three to four gigabytes, the CPU heap holds any CPU-only weights, and the iGPU draws its model and KV cache from the same pool through Level Zero, with zero-copy access possible.

7.2 Zero-copy and shared memory

The key advantage of unified memory is zero-copy: the CPU and iGPU read the same physical allocation without an intervening transfer. Through Level Zero this is a single `zeMemAllocShared` call, and OpenVINO performs it automatically in its GPU plugin since release 2023.2. The practical effect is that weight data does not have to be duplicated across a device boundary, which matters on a bandwidth-constrained part.

7.3 Allocation strategies for LLMs

For a model that exceeds the iGPU share, `llama.cpp` distributes layers between CPU and GPU through the `-ngl` flag, which sets how many layers go to the GPU while the rest run on CPU threads. The KV cache grows linearly with context length following

$$\text{KV bytes} = 2 \cdot L \cdot h_{kv} \cdot d_{\text{head}} \cdot n_{\text{ctx}} \cdot b,$$

which for Mistral 7B ($L = 32$, $h_{kv} = 8$, $d_{\text{head}} = 128$, $n_{\text{ctx}} = 4096$) amounts to about 536 MB in FP16 and 268 MB once quantised to INT8. Enabling INT8 KV-cache quantisation through OpenVINO GenAI therefore maximises the usable context.

7.4 Memory fragmentation

Fragmentation is the main risk over long sessions. Pre-allocating the maximum required memory avoids fragmenting re-allocations, fixed-size memory pools (as in vLLM's paged attention) keep the layout regular, and periodically restarting the OpenVINO runtime clears the arena on very long-running services.

8 Inference frameworks

8.1 `llama.cpp`

`llama.cpp` is the ultralight C++ engine that first made local LLM execution practical. Its SYCL backend (`ggml-sycl`) drives kernels onto the Iris Xe EUs through the oneAPI DPC++ toolchain, compiling to SPIR-V and dispatching over Level Zero. This is the native low-level Intel path, potentially the fastest, but the most delicate to configure [15]. Building it requires the oneAPI Base Toolkit and the `icx`/`icpx` compilers, and one flag must be left out of the `cmake` line for reasons explained below. The environment then has to be set as follows.

Table 8: Mandatory environment for the SYCL back-end on Iris Xe (empirically validated).

Variable	Value	Reason
`OLLAMA_NUM_GPU`	`-1`	Forces maximum layer offload; otherwise Ollama under-allocates
`OLLAMA_KV_CACHE_TYPE`	`f16`	`q8_0`/`q4_0` dequant bug in `ggml-sycl` yields NaN attention
`OLLAMA_FLASH_ATTENTION`	`false`	SYCL flash-attention kernel incomplete on Xe-LP
`SYCL_PROGRAM_COMPILE_OPTIONS`	`-cl-fp32-correctly-rounded-divide-sqrt`	Corrects exp/div in the JIT-compiled softmax
`shm_size` (Docker)	`8g`	Avoids fragmentation of Level Zero staging buffers

8.1.1 Documented SYCL bugs

EOS collapse from FP16 accumulation on QK-norm models. The `GGML_SYCL_F16=ON` CMake flag forces FP16 accumulation in every GEMM. Because it is baked into the SPIR-V kernels at compile time, the identically named environment variable has no runtime effect. Gemma 4 uses query–key normalisation with `attention_scale = 1.0` rather than the usual $1/\sqrt{d}$ scaling, so the pre-softmax logits sit in a wider numerical range that is sensitive to FP16 rounding. The failure chain is mechanical: FP16 accumulation loses precision at each of the 35 layers, the EOS logit eventually wins the argmax right after the `</think>` token, the model emits EOS immediately, and Ollama returns an empty-output error. Building without the flag, in FP32, restores coherent generation.

Table 9: SYCL accumulation precision on `gemma4:e2b` (Iris Xe).

Configuration	Decode (tok/s)	Output quality
SYCL FP16 (`GGML_SYCL_F16=ON`)	7.1	Empty response (EOS collapse)
SYCL FP32 (flag removed)	**6.02**	Coherent, drift-check passes

The FP32 build is barely slower than the broken FP16 one because the GPU GEMM is not the bottleneck. The 5.4 GB output projection runs on the CPU (the 4 GiB ceiling, Section 3.3), and the FP16/FP32 distinction only affects the 1.6 GB of transformer layers on the GPU. The measured 5.93 tok/s for this model in the benchmark (Section 10) confirms the 6.02 tok/s field figure.

The `lm_head` bottleneck. The initial SYCL allocation of the 5.4 GB output projection fails outright with `can't allocate 4697620480 Bytes`. Ollama's back-off then places the 35 transformer layers, the KV cache and the compute graph on the GPU (about 2.5 GB total) and leaves the 5.4 GB projection on the CPU. The result, reported as a 68/32 CPU/GPU weight split, is 35 of 36 layers on the GPU and about 6 tok/s, since every token waits on a CPU multiply over the $258\,048 \times \text{dim}$ matrix. No configuration removes this on a ggml back-end; only OpenVINO's tiled allocation does.

KV-cache and shared-memory bugs. The `q8_0` dequantisation kernel in the `ggml-sycl` revision used here does not flush subnormals correctly on Intel targets, which corrupts the attention matrices, so the KV cache must be pinned to `f16`. Separately, the default 64 MB Docker `/dev/shm` is too small for the Level Zero staging buffers under unified memory and causes random load-time instability, fixed by setting `shm_size: '8g'`.

8.1.2 Vulkan back-end

The Vulkan back-end is more portable and needs no oneAPI, but it is less stable on Intel iGPUs. Three settings are non-negotiable. Flash attention must be disabled (`OLLAMA_FLASH_ATTENTION=0`), since the Intel Mesa ANV implementation is experimental and silently corrupts output. The driver's buffer-object cache must be turned off (`ANV_BO_CACHE=0`) to avoid allocation failures, especially under Docker. And Gemma models must stay in Q4_K_M and under the Vulkan allocation limit, because a GPU/CPU split combined with Gemma's logit soft-capping produces endlessly repeated text. With these mitigations the back-end runs; the measured throughput appears in Section 10, where Vulkan is in fact faster than SYCL on most models, even if it remains the riskier choice on the Gemma family.

8.2 Ollama

Ollama is a high-level layer over `llama.cpp` that adds model management, a REST API and a simple CLI. Since release 0.4 it detects the Intel iGPU automatically through the Vulkan or SYCL back-end. It is the most convenient entry point, and the same environment caveats above apply when it drives the SYCL or Vulkan path.

8.3 Ollama-OV — the OpenVINO GenAI fork

Ollama-OV is Intel's fork of Ollama, hosted in `openvinotoolkit/openvino_contrib`, which replaces the `llama.cpp` back-end with the OpenVINO GenAI `LLMPipeline` while keeping Ollama's REST API and CLI [22]. It targets the CPU, the iGPU and the NPU through Level Zero, and consumes OpenVINO IR rather than GGUF.

Table 10: Ollama-OV against standard Ollama.

Aspect	Standard Ollama	Ollama-OV
Back-end	`llama.cpp` (native GGUF)	OpenVINO GenAI `LLMPipeline`
Primary format	GGUF	OpenVINO IR (`.xml`+`.bin`, packed as `.tar.gz`)
GGUF support	Native	Experimental (via OpenVINO GenAI)
Target accelerator	CPU / GPU (Vulkan, SYCL)	CPU / iGPU / NPU (Level Zero)
Modelfile keys	—	`ModelType`, `InferDevice`

The fork adds three Modelfile keys. `ModelType "OpenVINO"` declares the format, `InferDevice` selects `GPU` (default), `CPU` or `NPU`, and `stop_id`/`max_new_token` tune generation. Without an explicit device the fork prefers the GPU and falls back to the CPU. Importing a model means downloading the pre-converted IR, packing it into a `.tar.gz`, and registering it with `ollama create`. The library of validated IR models spans the Qwen3, DeepSeek-R1-Distill, Llama 3.2, Phi-3 and Gemma-2 families, each published under a compression scheme: `INT4_SYM_128` (similar to GGUF Q4_0), `INT4_SYM_CW` (channel-wise, optimal for the NPU), or `INT4_ASYM` (better accuracy but CPU/GPU only, unsupported on the NPU).

Two practical caveats apply. GGUF loading is marked experimental and limited to Q4_K_M on the Qwen2.5, Llama 3.x and DeepSeek-R1-Distill architectures. And building from

source against OpenVINO GenAI ≥ 2025.4.1 requires fixing a typo in `genai/genai.go`, where the constant `OV_GENAI_STREAMMING_STATUS_RUNNING` (double M) must become `OV_GENAI_STREAMING_STATUS_RUNNING`; the pre-built binaries, which target 2025.3.0, are unaffected.

8.4 OpenVINO GenAI (direct SDK)

The OpenVINO GenAI SDK is the official Intel path and gives the best performance on the Intel iGPU and NPU. A model is converted once with `optimum-cli export openvino` (here at INT4 with `group_size=128` and an AWQ pass), and inference is a few lines around an `LLMPipeline` bound to `device="GPU"`, with a lambda streamer printing tokens as they arrive. It underlies both Ollama-OV and the model server discussed below.

8.5 A custom OpenAI-compatible server for Gemma 4

> This server was the first workaround to the `Unsupported 'gemma4' VLM model type` error raised by the model server. It is documented and validated on CPU. A cleaner solution on the Iris Xe iGPU was found later through OpenVINO graph stitching (Section 8.6); the present section remains relevant when vision input is required.

The natural route, serving `OpenVINO/gemma-4-E4B-it-int4-ov` through the model server, fails because Gemma 4 is a vision-language model (`Gemma4ForConditionalGeneration`) that needs `OVModelForVisualCausalLM`, a class absent from the stable optimum-intel branch in mid-2026. I built a FastAPI server (Python 3.11, port 8000) on the development branch `rkazants/optimum-intel` [17], the only source of that class for Gemma 4 [13]. The server exposes an OpenAI-compatible `/v1/chat/completions` endpoint with Server-Sent Events streaming, alongside Ollama-compatibility and Prometheus endpoints, and serialises inference behind a one-slot semaphore with a four-deep queue, returning HTTP 503 when the queue is full. Streaming bridges a `TextIteratorStreamer` on a background thread to an `asyncio.Queue` that feeds the SSE chunks.

Table 11: Critical dependencies of the custom server.

Component	Version	Reason
OpenVINO Runtime	2026.1.0	Compatible with `OVModelForVisualCausalLM`
optimum-intel	`rkazants@eac3893`	Branch `support_gemma_4`; only source of the VLM class
transformers	5.5.0	Required by the branch; incompatible with 5.6+
Base image	`openvino/ubuntu22_runtime:2026.1.0`	Runtime preinstalled

The device configuration matters. On the GPU the server keeps the KV cache in FP16, disables on-the-fly dynamic quantisation, and pins an FP32 inference-precision hint for the stability of Gemma 4's QK-norm. On a multi-socket CPU it uses a single stream with NUMA affinity, and the thread count must match the physical cores of one socket rather than the total: on the dual-Xeon E5-2643 v2 conversion server (two sockets, 24 vCPUs, 122 GiB RAM, no Intel GPU) twelve threads were optimal, while using all 24 vCPUs cost about 30% of throughput to cross-NUMA latency. A dummy inference at start-up triggers OpenVINO's JIT compilation and warms the SPIR-V cache, cutting the first real request's TTFT from several seconds to about 40 ms.

Measured performance and what is extrapolated. The custom server was benchmarked on the conversion server's CPU only, since that machine has no Intel GPU and is distinct from the Iris Xe test laptop. That CPU figure is a floor, not a representative iGPU result.

Table 12: Custom-server performance: one measured point, the rest extrapolation.

Device	Decode (tok/s)	Status
Xeon E5-2643 v2, 12 threads, NUMA	**1.9**	**Measured** (this work)
Iris Xe 96 EU (OVMS path)	14.8 (E4B)	Measured for the stitched model, Section 10
Arc Meteor Lake	15–25	Theoretical, bandwidth-based; hardware not owned, not measured
Arc Lunar Lake (Xe2)	25–40	Theoretical, bandwidth-based; hardware not owned, not measured

The earlier draft of this report quoted a blanket 25–40 tok/s for OpenVINO on Iris Xe. The measured data does not support that range for this hardware: it holds only for the smallest models, while 7B models sit near 12–14 tok/s (Section 10). The single-slot limit (`MAX_CONCURRENT_REQUESTS=1`) follows from the non-thread-safety of the VLM class in the custom branch and is the main reason to prefer the model-server path wherever vision is not needed.

8.6 Serving Gemma 4 on Iris Xe through OpenVINO graph stitching

Result. `bash gemma.sh` serves `gemma-4-E4B-it-int4-ov` on the Iris Xe iGPU through the OpenVINO Model Server [11] (image `openvino/model_server:2026.2-gpu`) at `localhost:8000`, with coherent responses and a working stateful KV cache [16]. This replaces the custom server of Section 8.5 for serving on Intel hardware.

The standard vision-language export of Gemma 4 produces a fragmented IR in four parts: token embeddings, per-layer embeddings (PLE), the stateful decoder, and the vision tower. The decoder expects an external `per_layer_inputs` tensor that the PLE sub-graph computes and an orchestrator injects, which the optimum-intel Python runtime does. The model server's C++ continuous-batching pipeline performs no such orchestration: it assumes a single self-contained causal-LM graph with the interface (`input_ids`, `attention_mask`, `position_ids`, `beam_idx`) → `logits`. Loading the fragmented export therefore fails at the first inference with `Add_213682 Argument shapes are inconsistent`, where `Add_213682` is a node the pipeline synthesises at runtime and which is absent from the `.xml` files, so patching the XML has no effect.

Two alternatives were ruled out. Patching tensor shapes in the XML cannot work, since the offending node is synthesised at runtime. Re-exporting as a text-only `OVModelForCausalLM` fails because `Gemma4ForCausalLM.from_pretrained` randomly initialises five critical weights stored under `model.language_model.*`, and even after copying them across and tying weights, the export trace breaks on the KV-sharing plus sliding-window combination with a tensor-size mismatch. Reaching into the library internals was judged too risky.

The stitching solution. The reliable approach reuses the correct fragmented IR, including its validated INT4 weights, and wires the embedding sub-graphs into the decoder at the OpenVINO level, with no re-tracing and no re-quantisation.

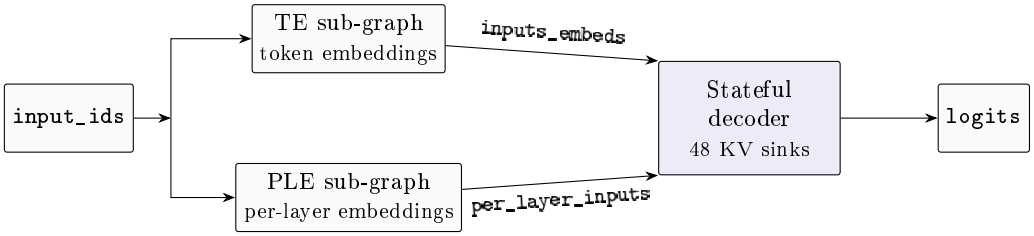


Figure 3: Graph stitching: the TE and PLE sub-graph outputs are rewired onto the decoder's internal inputs, yielding one stateful graph with the standard model-server interface.

```python
import openvino as ov
from pathlib import Path

def stitch_single_model(split_dir: Path, out_xml: Path):
    core = ov.Core()
    te  = core.read_model(str(split_dir / "openvino_text_embeddings_model.xml"))
    ple = core.read_model(str(split_dir / "openvino_text_embeddings_per_layer_model.xml"))
    lm  = core.read_model(str(split_dir / "openvino_language_model.xml"))

    # One shared input_ids feeds both TE and PLE
    shared_in = te.get_parameters()[0]
    shared_in.set_friendly_name("input_ids")
    shared_in.output(0).set_names({"input_ids"})
    for consumer in list(ple.get_parameters()[0].output(0).get_target_inputs()):
        consumer.replace_source_output(shared_in.output(0))

    # Wire TE/PLE outputs onto the decoder's internal inputs
    te_out  = _result_source(te)
    ple_out = _result_source(ple)
    for consumer in list(_param_by_name(lm, "inputs_embeds").output(0).get_target_inputs()):
        consumer.replace_source_output(te_out)
    for consumer in list(_param_by_name(lm, "per_layer_inputs").output(0).get_target_inputs()):
        consumer.replace_source_output(ple_out)

    # Keep the standard model-server causal-LM interface, preserve KV-cache sinks
    keep = [_param_by_name(lm, n) for n in ("attention_mask", "position_ids", "beam_idx")]
    merged = ov.Model(lm.get_results(), lm.get_sinks(),
                      [shared_in] + keep, name="gemma4_text_merged")
    ov.save_model(merged, str(out_xml))
```

The output is a single `openvino_model.xml` exposing the standard interface and preserving the 48 stateful KV-cache sinks (the `ReadValue`/`Assign` pairs) that continuous batching requires. The conversion pins the Hugging Face revision `d6436b3d62967e1af08bbb046c6300b2a9ae8e85`, rewrites `config.json` to `model_type=gemma4_text`, and copies the tokenizer and chat template. The merged model is 6.2 GB in INT4, and the vision tower is dropped, so the served model is text-only.

Validation. The server loads the model as a continuous-batching servable, reaches the `AVAILABLE` state, and lists it at `/v3/models`. A one-word capital-of-France prompt returns *Paris*, and a haiku prompt returns a coherent multi-token poem, which confirms that the stateful KV cache works across decoding steps. The generated `graph.pbtxt` shows `device: "GPU"`, so inference runs on the Iris Xe in unified memory. The measured throughput, 14.8 tok/s for the E4B variant and 24.5 tok/s for E2B, is reported in Section 10.

Three pitfalls cost time during conversion. Saving the 6.2 GB decoder triggered a SIGBUS (`exit 135`) that truncated the file mid-write, so the base decoder must be restored from an intact copy rather than re-saved. A missing Hugging Face token in a non-interactive SSH session made `from_pretrained` fail, fixed by a warn-and-continue check together with `HF_HUB_OFFLINE=1` against the local cache. And an incomplete snapshot left weights missing, which is why the full revision is pinned on every download call.

Table 13: Custom server (Section 8.5) against model-server stitching.

Criterion	Custom FastAPI	OVMS + stitching
Concurrency	One request (semaphore)	Continuous batching (N sequences)
Infrastructure	Custom code (≈1355 lines)	Standard server, no serving code
Custom branch	Yes (`rkazants/optimum-intel`)	No (standard `OVModelForCausalLM` after stitching)
Conversion	None (direct HF model)	Yes (stitching pipeline, ≈30 min on the GPU box)
Iris Xe validated	CPU only	**Validated on Iris Xe**
Vision support	Yes (VLM pipeline)	No (vision tower removed)
Recommended for	Vision and reasoning, CPU	**Production iGPU serving, maximum throughput**

8.7 IPEX-LLM

> **Archived on 28 January 2026.** The `intel/ipex-llm` repository is no longer maintained: no new releases, no security patches, no compatibility updates. It is kept here for documentation and for systems that already deploy it.

IPEX-LLM, formerly BigDL-LLM, was a PyTorch layer providing additional SYCL kernels and a turnkey patched Ollama binary. Against upstream `ggml-sycl` it offered native weight-only INT4 with calibration, hardware-native INT8 KV cache, paged attention through a vLLM bridge, and a UMA-aware allocator. I validated it on Gemma 2 2B with the `intelanalytics/ipex-llm-inference-cpp-xpu` image, which detected the Iris Xe and offloaded all 27 of 27 layers, using about 1.5 GB of shared memory with coherent output. I did not benchmark its decode throughput, so no tokens-per-second figure is claimed for it here.

The full offload is consistent with the 4 GiB ceiling rather than a counter-example to it. Gemma 2 2B has a far smaller vocabulary than Gemma 4, so its combined output projection and embeddings fit under 4 GiB and the whole model goes to the GPU. Only models with a vocabulary above roughly 200k tokens, such as Gemma 4 at 258k, are affected. Since the archival, the recommended replacements are `llama.cpp` SYCL FP32 for the GGUF path on the iGPU and Ollama-OV for maximum performance and stability.

8.8 `ollama-intel-arc` — an integrated community stack

`ollama-intel-arc` [5] orchestrates a full Intel-GPU AI stack through Podman Compose: Ollama on the IPEX-LLM image, a preconfigured Open WebUI, ComfyUI and SD.Next for image generation, and Whisper running on `-device xpu` for transcription. It is the most turnkey documented option for Intel Arc on Linux and remains functional while the IPEX-LLM base image stays available, though it inherits that image's frozen, unmaintained status. The project was tested by its author on a Core Ultra 7 155H with Arc Graphics (Meteor Lake-P, 128 EU) on Fedora 41; its behaviour on Iris Xe 96 EU is not documented there, though the separate Gemma 2 2B validation above covers that part. Two operational notes carry over: Whisper needs the explicit `-device xpu` flag or it falls back silently to the CPU, and the bundled Open WebUI disables authentication, so it should not be exposed on an untrusted network.

8.9 Framework comparison

The table below consolidates the back-ends on the Iris Xe target. Throughput figures point to the measured results of Section 10; ranges marked *est.* are not measured on this hardware.

Table 14: Inference back-ends on Iris Xe. Measured throughput is detailed in Section 10.

Back-end	NPU	Full offload	Gemma stability	Decode (tok/s)
CPU only	No	—	Always correct	3–8
`llama.cpp` Vulkan	No	9/36 layers	DP4A bug, fragile	see §10
`llama.cpp` SYCL FP16	No	35/36	EOS collapse	7.1 (broken)
`llama.cpp` SYCL FP32	No	35/36	Stable	6.0 (gemma4:e2b)
Ollama-OV	**Yes**	Full (OV)	Stable	**≈10–39, size-dependent**
IPEX-LLM (archived)	No	27/27 (2B)	Stable	not benchmarked
OpenVINO GenAI SDK	Yes	Full	Stable	≈10–39, size-dependent
Custom FastAPI (VLM)	No	Full (OV)	Stable, vision	1.9 CPU; iGPU via OVMS
OVMS + stitching	No	Full (OV)	Stable (text)	**see §10**

8.10 Choosing a back-end

The decision reduces to a few questions. A project that must stay within the Ollama and GGUF ecosystem, and needs the best stability on Gemma 4 or other QK-norm models, should use `llama.cpp` SYCL built in FP32 without `GGML_SYCL_F16`, accepting about 6 tok/s from the CPU-bound output projection and a roughly twenty-minute build with the mandatory environment. The archived IPEX-LLM path should be migrated away from. A project free of the GGUF requirement and oriented to performance should use Ollama-OV or, for production serving with continuous batching, the model server, both of which clear the 4 GiB ceiling through OpenVINO and deliver the size-dependent throughput of Section 10 at the cost of an IR conversion step. Vulkan must not be used for Gemma 4 on Iris Xe, where the DP4A bug and the allocation ceiling together drop it to nine of thirty-six layers with corrupted output, uncorrectable without a driver patch. For the specific case of Gemma 4 and other PLE models on the model server, graph stitching (Section 8.6) is the production answer on the iGPU, and the custom server (Section 8.5) is the fallback when vision is required.

8.11 Image diffusion

For image generation, Automatic1111 and ComfyUI reach the Intel iGPU through DirectML on Windows or Vulkan, while the Hugging Face Diffusers library combined with optimum-intel offers an official OpenVINO pipeline. A Stable Diffusion model is loaded through `OVStableDiffusionPipeline` with `export=True`, reshaped to the target resolution, and compiled for the GPU. This path was not benchmarked here and is noted as future work.

9 Integration and deployment pipeline

9.1 Pipeline architecture

The deployment pipeline runs in four stages. A source model is acquired from the Hugging Face Hub, converted to OpenVINO IR or GGUF, quantised to INT4 or INT8, and validated for quality before it reaches the runtime. The runtime then dispatches across the CPU, the iGPU and the NPU, and exposes the model through a REST API, a Python SDK, a graphical front-end or a command line. The conversion and quantisation stages are where most of the engineering effort sits, since they determine both the memory footprint and whether the model clears the 4 GiB ceiling.

9.2 Conversion steps

Acquisition is a `snapshot_download` that ignores the Flax and msgpack artefacts. Conversion and quantisation are a single `optimum-cli export openvino` [10] invocation that sets the task to

`text-generation-with-past`, the weight format to INT4 with `group_size=128` and an AWQ pass with scale estimation. Quality is then checked against the `lm-evaluation-harness` on HellaSwag, ARC and WinoGrande, after which the performance benchmark of Section 10 runs. Deployment is a local OpenAI-compatible server, either the model server directly or a thin FastAPI layer over an `LLMPipeline`.

9.3 Conversion methods in practice

In practice the models in this study reached the OpenVINO format along three distinct paths, chosen by what the model already offered upstream. All conversions ran on the dual-Xeon server described in Section 8.5, CPU-only, since NNCF INT4 quantisation is CPU-bound there and that machine carries the 122 GiB of RAM the export needs; the converted artefacts were then copied to the Iris Xe laptop for serving.

The first path, used for any standard text model that already has a pre-converted `OpenVINO/<name>-int4-ov` repository, is no local export at all: the model server downloads and serves the published IR directly. The second path is the single-file graph stitching of Section 8.6, required for the Gemma 4 PLE checkpoints whose split vision-language export the server cannot load; it produces one self-contained `openvino_model.xml` that retains `ScaledDotProductAttention` and the KV-cache sinks. The third path is a plain `optimum-cli export openvino` at INT4 (`sym=False`, `group_size=128`, `ratio=1.0`) for standard architectures that have no pre-converted repository.

Table 15: Conversion path taken by each model in this study.

Path	Models
A — pull pre-converted IR	Qwen2.5-1.5B, Qwen2.5-7B, Phi-3-mini, Mistral-7B, Gemma-2-9b (and the published Qwen3 variants)
B — stitched single-file export	Gemma-4-E2B, Gemma-4-E4B
C — `optimum-cli` INT4 export	Qwen2.5-3B

Two limits shaped what could be converted. The gated Llama 3.2, Llama 3.1 and Gemma-2-2b checkpoints returned a `403 GatedRepoError` because the export account had not accepted the Meta and Google licences, so they were left for a later round; the ungated Qwen2.5-3B converted without trouble. The three models converted in this session — the two stitched Gemma 4 variants and Qwen2.5-3B — were then published as public INT4 OpenVINO repositories with model cards, the Gemma 4 cards documenting the stitching fix.

9.4 Retrieval-augmented generation

For professional use over a local knowledge base, a retrieval-augmented generation (RAG) stack keeps everything on the machine. Documents are embedded by a small model such as `BAAI/bge-small-en-v1.5` (33M parameters, fast on CPU or NPU), stored in a local ChromaDB or FAISS index, and retrieved by similarity; the top-k passages are then passed to the LLM running on the iGPU, typically Mistral 7B or Phi-3 Medium in INT4. ChromaDB on SQLite needs no server, and LangChain or LlamaIndex with an OpenVINO back-end ties the stages together.

10 Benchmarking methodology and measured results

10.1 Test environment

Every figure in this section comes from a single laptop, described below [14]. The Arc figures elsewhere in the report are vendor specifications or bandwidth-based extrapolations and were not measured.

Table 16: Test machine for all measured results.

Parameter	Value
iGPU	Intel Iris Xe Graphics, 96 EU (Raptor Lake-U), Xe-LP
Host CPU	Intel Core i7-1365U (10 cores / 12 threads)
System RAM	30 GiB LPDDR4X-4267, shared with the iGPU
Operating system	Arch Linux
GPU driver	Mesa ANV (Vulkan) / Level Zero (SYCL, OpenVINO)
Model server	`openvino/model_server:2026.2.1-gpu`

10.2 Metrics and protocol

Two metrics carry the analysis. Time to first token (TTFT) is the latency from prompt submission to the first token, dominated by prompt processing and, on a cold model, by JIT compilation. Decode throughput, in tokens per second, is the autoregressive generation rate once the prompt is processed, and on this bandwidth-bound hardware it is set by the weight bytes read per token. The protocol was held fixed across back-ends: a 174-character prompt, between 230 and 256 generated tokens, and three runs per configuration, on the single Iris Xe machine. The GGUF models ran in Q4_K_M, the OpenVINO IR models in INT4 with `group_size=128`, and the two Gemma 4 entries on the model server are the stitched graphs of Section 8.6.

10.3 Cross-back-end results

Table 17: Measured decode throughput (tok/s, mean of three runs) on Intel Iris Xe 96 EU. SYCL and Vulkan run GGUF Q4_K_M through Ollama; OVMS runs OpenVINO IR INT4. A dash means the configuration was not run.

Model	Params	SYCL	Vulkan	OVMS
Llama 3.2 1B	1.2B	17.59	16.86	—
Qwen2.5 1.5B	1.5B	12.10	23.37	**39.27**
Gemma 2 2B	2.6B	12.52	13.57	—
Gemma 4 E2B	∼2B	5.93	—	**24.46**
Qwen2.5 3B	3.1B	7.06	12.47	26.06
Llama 3.2 3B	3.2B	8.23	12.33	—
Phi-3 mini	3.8B	10.90	12.23	22.19
Qwen3 4B	4.0B	—	—	13.48
Gemma 4 E4B	∼4B	4.39	8.22	**14.76**
Mistral 7B	7.2B	3.93	6.08	**14.03**
Qwen2.5 7B	7.6B	3.93	6.11	12.18
Llama 3.1 8B	8.0B	3.72	5.67	—
Qwen3 8B	8.2B	—	4.24	10.90
Gemma 2 9B	9.2B	4.47	4.62	9.88

Three readings stand out. First, the OpenVINO Model Server is fastest at every size that was run on it, and the gap widens with the model. For Mistral 7B it reached 14.03 tok/s against 6.08 over Vulkan and 3.93 over SYCL, that is 2.3 times Vulkan and 3.6 times SYCL. The reason is the 4 GiB ceiling: the GGUF back-ends strand the output projection on the CPU, while OpenVINO's tiled allocation keeps the whole model on the iGPU. Second, the stitched Gemma 4 path delivers real performance, 24.46 tok/s for E2B and 14.76 for E4B, the E2B figure being 4.1 times the 5.93 tok/s that SYCL manages on the same model.

Third, among the GGUF back-ends Vulkan is generally faster than SYCL on this hardware, despite its fragility on the Gemma family, which is why SYCL FP32 is the stability choice rather than the speed choice.

The 7B result also validates the bandwidth analysis of Section 3.3. The back-of-envelope ceiling for a 7B Q4 model on a 57–68 GB/s part is about 16 tok/s, and the measured 14 tok/s on the model server sits just under it, which confirms that decode on this hardware is bandwidth-bound and that OpenVINO is close to extracting the available bandwidth. Across all configurations TTFT stayed below one second, between 0.10 and 0.51 s, with the warm OpenVINO models at the low end, so first token latency is not a limiting factor for an interactive workload on short prompts.

10.4 Quality methodology

Output quality was assessed qualitatively, by checking for the corruption modes described in Section 8 — empty responses, repetition loops, incoherent text — before any throughput figure was recorded. A quantitative quality evaluation was not carried out: the perplexity routine was left as a stub, so no perplexity is reported as measured. For reference, the literature places Mistral 7B near 5.25 perplexity on WikiText-2 in FP16, rising to about 5.46 in Q4_K_M and 7.12 in Q2_K, which illustrates the expected degradation pattern without standing as a measurement from this work. A full evaluation on HellaSwag, ARC, MMLU and WinoGrande through the `lm-evaluation-harness` is left as future work (Section 13.5).

11 Thermal behaviour and power

11.1 Thermal profile under inference

LLM inference is a sustained load, unlike the short bursts of a game, and it produces a characteristic thermal curve. The SoC warms over the first thirty seconds to roughly two minutes, settles into a working range around 75 °C, and on a thin chassis can climb past 85 °C into throttling on a long generation. Once the junction temperature reaches the 75–90 °C band, frequencies drop automatically, so any session longer than a few minutes operates in this throttled steady state rather than at peak clocks.

11.2 Throttling mitigation

Neither OpenVINO nor `llama.cpp` controls GPU frequency directly; that is handled at the operating-system level. On Linux, `intel-gpu-tools` provides monitoring and `powercap` caps the TDP through RAPL, while on Windows the Intel Dynamic Tuning Technology serves the same role. For very long generations, splitting the work into chunks with short cool-down pauses keeps the part out of progressive throttling, at a modest cost in wall-clock time.

11.3 Energy and battery

Energy efficiency, expressed as tokens per joule, is the metric that matters on battery, and it follows from the decode rate divided by the power draw. Power was not measured through RAPL in this work, so the efficiency and battery figures here are estimates that combine the measured throughput with an assumed power envelope. On that basis, Gemma 4 E4B served through the model server at the measured 14.8 tok/s and an estimated 15 W would sit near 1.0 tok/J, against roughly 0.5 tok/J for the SYCL FP32 path at about 6 tok/s. For a 50–70 Wh battery, a sustained chatbot session would plausibly cut runtime by 40 to 50 % relative to web browsing, but these numbers depend on the configured TDP, the thermal state and the request profile, and they should be confirmed with direct RAPL measurement. The throughput components are measured; the wattages are not.

Three optimisations reduce the energy cost regardless of the exact figures. Early stopping ends generation at the EOS token instead of producing empty tokens, idle unloading frees the iGPU after a period of inactivity, and a mixed schedule that keeps short prompts on the CPU and reserves the iGPU for long generations avoids spinning up the GPU for sub-second requests.

12 Target use cases

12.1 Local conversational assistant

The first use case is a local replacement for a cloud chatbot. A practical setup pairs Mistral 7B or Phi-3 Medium in Q4_K_M with Open WebUI over the Ollama or model-server API, on a 16 GB Iris Xe machine. The measured behaviour on the test laptop, for Gemma 4 E4B served through the stitched model server, is a warm TTFT near 40 ms and a decode rate of 14.8 tok/s, well above the SYCL FP32 figure of about 6 tok/s for the same model and comfortably within reading speed, with coherent output and a working reasoning mode.

12.2 Local code assistant

A Copilot-style assistant that never sends source code off the machine pairs Qwen2.5-Coder-7B in INT4 with the Continue.dev plugin for VS Code or JetBrains over an Ollama back-end, at an 8192-token context to hold the open files and recent history. On the throughput evidence of Section 10, a 7B coder model on the model server would generate at roughly 12 tok/s on this hardware, which is usable for inline completion.

12.3 Document retrieval

For querying a local document set — contracts, course notes, technical manuals — the RAG stack of Section 9 applies, with a multilingual `bge-m3` embedder in INT8, a local ChromaDB index, and Mistral 7B or Phi-3 Medium in INT4 behind a self-hosted interface such as AnythingLLM or Khoj.

12.4 Use cases left for later

Several directions are out of scope here because the hardware or the models were not tested. Speech transcription with Whisper on `-device xpu` is documented as working on Arc through the community stack and can target the NPU through OpenVINO, but was not run on the Iris Xe laptop. Image generation with ComfyUI and the OpenVINO plugin, and INT8 computer-vision models such as YOLOv8 on the NPU, are likewise documented elsewhere and noted as future work in Section 13.5.

13 Limits, risks and future work

13.1 Hardware limits

The dominant limit is memory bandwidth, at 68–120 GB/s against 500 GB/s and more for GDDR6X, which caps models above 7B. The absence of dedicated VRAM puts inference in competition with the rest of the system for the shared pool, and thermal variability means a benchmark depends on ambient temperature, chassis and battery state. The NPU is doubly constrained: it is restricted to topologies fixed at deployment, and it is structurally ill-suited to autoregressive decoding. Optimised for static shapes, the NPU compiler requires all tensor shapes at compile time, whereas a growing KV cache is dynamic, so output becomes unstable beyond roughly one to two thousand tokens of context and the bandwidth-to-TOPS ratio is unfavourable in decode, where the constraint is weight reads rather than compute. The

NITRO framework and the `NPUW_LLM_PREFILL_CHUNK_SIZE` parameter mitigate this only partly. Finally, the 4 GiB allocation ceiling on Xe-LP (Section 3.3) keeps any matrix above 4 GB off the GPU under Vulkan, SYCL and OpenCL, and is cleared only by OpenVINO's tiled allocation; it is absent on Arc Xe-HPG and Xe2.

13.2 Software limits

The ecosystem is fragmented, with SYCL, Vulkan, OpenCL and DirectML coexisting and not always agreeing across configurations. The most performant back-end, SYCL, needs oneAPI and is better integrated on Linux than on Windows. Pre-12th-generation iGPUs have limited or absent OpenVINO and Level Zero support, and recent architectures such as Mamba and RWKV are not yet optimised for the Intel iGPU. The model server cannot serve the fragmented vision-language export of Gemma 4 unmodified, a limit resolved here by graph stitching (Section 8.6).

13.3 Vision-language models on the model server

Gemma 4 illustrates a structural tension between how the model server expects a model and how a vision-language architecture is exported. The export is several sub-graphs — token embeddings, per-layer embeddings, the decoder, and a SigLIP vision tower — whereas the server expects one self-contained causal-LM graph. Stitching reconciles them, but at a cost: the vision tower is dropped, so the served model is text-only.

Table 18: What graph stitching preserves and what it sacrifices for Gemma 4 on the model server.

Capability	Direct VLM export	After stitching
Text generation	Yes (Python runtime)	Yes (continuous batching)
Image input	Yes (SigLIP)	No (vision tower removed)
Multi-request serving	No (single slot)	Yes (continuous batching)
Standard deployment	No (custom code)	Yes (standard server)

No satisfactory text-and-vision path exists through the model server for Gemma 4 in production today. A complete solution would require a multi-graph orchestrator in the server's C++ pipeline, or an IR format that folds the vision encoder into a single graph.

13.4 Risks and the consolidated bug table

The main operational risks are an Intel GPU driver regression, an OpenVINO API change, a GGUF format update, non-reproducible throttling and memory leaks over long sessions; pinning the driver and the runtime in production, tracking changelogs with a regression suite, and restarting the inference server periodically address most of them. The numerical-corruption risks specific to this hardware, together with their fixes, are gathered below.

Table 19: Back-end bugs observed on Iris Xe and their resolution.

Bug	Back-end	Severity	Resolution
DP4A bit-alignment (Q4_K_M)	Vulkan	Critical	Driver patch; avoid Vulkan
4 GiB single-buffer ceiling	Vulkan, SYCL, OpenCL	Architectural	OpenVINO tiled allocation
`GGML_SYCL_F16` EOS collapse	SYCL	Critical	Drop the flag; build FP32
KV cache `q8_0` dequant NaN	SYCL	High	`OLLAMA_KV_CACHE_TYPE=f16`
Flash-attention device lost	Vulkan	High	`OLLAMA_FLASH_ATTENTION=false`
Flash-attention kernel incomplete	SYCL	High	`OLLAMA_FLASH_ATTENTION=false`
`ext_intel_free_memory` unsupported	SYCL	Medium	`ZES_ENABLE_SYSMAN=1` (partial)
NPU instability past 2k tokens	NPU	Medium	Chunk prompts, limit context
`STREAMMING` typo	Ollama-OV (source)	Build	Fix before building against GenAI $\geq$ 2025.4.1

13.5 Future work

Several directions follow naturally. On quantisation, BitNet b1.58 [20] trains models natively at 1.58 bits and could be transformative for the edge if it proves feasible on the Intel iGPU, and the AWQ and GPTQ calibrations left unused here are worth evaluating. On kernels, the XeTLA template library would allow low-level GEMM tuning inside `llama.cpp`, and OpenVINO's AUTO mode invites a study of optimal CPU–iGPU–NPU layer scheduling. On context, RoPE scaling and YaRN extend the window without retraining and their effect on iGPU throughput is unmeasured, while QLoRA on-device fine-tuning [4] would enable local personalisation within a few gigabytes.

The most concrete software gaps concern the OpenVINO serving stack. Adding the `ov_genai_vlm_pipeline` to Ollama-OV would unlock vision models such as LLaVA, Llama 3.2 Vision and Gemma 4 through the standard Ollama interface, and it is the declared priority of the project. Merging `OVModelForVisualCausalLM` into stable optimum-intel would remove the dependency on a pinned commit and resolve the model server's `Unsupported 'gemma4'` error at the source. A native PLE orchestrator in the model server's continuous-batching pipeline would remove the need for manual stitching for every new PLE architecture, and an `OVInferRequest` pool would lift the single-slot limit of the custom server for multi-user serving. An automatic Hugging-Face-to-IR conversion in Ollama-OV would close the last usability gap relative to `ollama pull`.

14 Conclusion

This study set out to determine, by measurement, what it takes to run generative AI on the Intel iGPU of a laptop, and it answers on three levels. On hardware, the progression from Iris Xe at 68 GB/s to Lunar Lake at 120 GB/s is enough to make 7B inference viable in daily use, and the NPU opens a path for light models on recurring tasks, even if it remains ill-suited to autoregressive decoding. On software, the ecosystem has settled around two poles: OpenVINO for an Intel-native, high-performance but more technical approach, and `llama.cpp` with Ollama for maximum accessibility at a lower ceiling, with the two converging through OpenVINO GenAI and optimum-intel. On models, INT4 is the balance point, and the measured results show that the OpenVINO path is the one that makes 7B-class models comfortable on this hardware, at 12–14 tok/s against about 6 over Vulkan and below 4 over SYCL.

Two findings are specific to this work. The 4 GiB allocation ceiling of Xe-LP, rather than raw compute, is what throttles every GGUF back-end on large-vocabulary models, and OpenVINO's tiled allocation is the only user-space escape from it. And serving Gemma 4 on the iGPU, blocked by its fragmented vision-language export, was made to work through graph stitching that rebuilds a single stateful decoder the model server can run, at a measured 14.8 tok/s for the E4B variant. Memory bandwidth sets the ceiling throughout: the measured 7B throughput sits just under the bandwidth-derived bound, which both explains the numbers and marks the limit that only future LPDDR6 and on-package memory will

move. The under-studied Intel iGPU, present in hundreds of millions of machines, deserves attention in proportion to its weight in the democratisation of sovereign, on-device AI.

15 Glossary

15.1 Acronyms

AMX	Advanced Matrix Extensions (Intel CPU)
ASR	Automatic Speech Recognition
AWQ	Activation-aware Weight Quantization
bpw	bits per weight
cTDP	configurable Thermal Design Power
DP4A	Dot-Product of 4 elements, Accumulate (INT8)
DVMT	Dynamic Video Memory Technology
EU	Execution Unit (Intel GPU)
GGUF	GPT-Generated Unified Format (`llama.cpp`)
GPTQ	Gradient-based Post-Training Quantization
iGPU	integrated Graphics Processing Unit
IR	Intermediate Representation (OpenVINO `.xml`/`.bin`)
KV	Key/Value (attention cache)
LLM	Large Language Model
NNCF	Neural Network Compression Framework
NPU	Neural Processing Unit
OV	OpenVINO
OVMS	OpenVINO Model Server
PLE	Per-Layer Embeddings (Gemma-3n / Gemma 4)
PTQ	Post-Training Quantization
QK-norm	Query–Key normalisation
RAG	Retrieval-Augmented Generation
RAPL	Running Average Power Limit
SLM	Small Language Model
SoC	System on a Chip
SSE	Server-Sent Events
STT	Speech-to-Text
SYCL	C++ heterogeneous programming standard (oneAPI)
TDP	Thermal Design Power
TE	Token Embeddings
TOPS	Tera-Operations Per Second
TPS	Tokens Per Second
TTFT	Time To First Token
UMA	Unified Memory Architecture
VLM	Vision-Language Model
VNNI	Vector Neural Network Instructions
XMX	Xe Matrix Extensions (Intel GPU)

15.2 Definitions

4 GiB allocation ceiling

The architectural limit of the Xe-LP command streamer, which is 32-bit and therefore cannot address a single buffer larger than 4 GiB. Any matrix above that size must remain on the CPU under Vulkan, SYCL or OpenCL.

Continuous batching

A serving strategy that processes several requests of differing sequence lengths together, keeping the accelerator busy. It requires a single stateful causal language-model graph.

Decode throughput

The number of tokens generated per second during the autoregressive phase, once the prompt has been processed. On bandwidth-bound hardware it is governed by how many bytes of weights must be read for each token.

Graph stitching

Rewiring exported OpenVINO sub-graphs at the runtime level so that the embedding outputs feed the decoder's internal inputs, yielding a single self-contained graph without re-tracing or re-quantising the model.

KV cache

The cache of attention keys and values that avoids recomputing past tokens at every decoding step. Its size grows linearly with context length and can itself be quantised.

Layer offloading

The distribution of a model's transformer layers between the GPU and the CPU when the model does not fit entirely on the accelerator. Throughput is then limited by the slowest device in the split.

Logit soft-capping

A bounding of the output logits, $f(x) = \tanh(x/c) \cdot c$, used by the Gemma family. It is sensitive to the precision of intermediate computations and can corrupt generation on numerically fragile back-ends.

Output projection (`lm_head`)

The final linear layer that maps the hidden state onto the vocabulary. For a 256k-token vocabulary it can exceed 4 GiB on its own, which makes it the first casualty of the allocation ceiling.

Per-Layer Embeddings (PLE)

An auxiliary embedding tensor introduced by the Gemma-3n and Gemma 4 architectures, computed by a separate sub-graph and injected into the decoder. It is the reason a standard serving pipeline cannot load the model unmodified.

Quantisation

Reduction of the numerical precision of a model's weights, and optionally its activations, from a wide format such as FP32 (four bytes per value) to a compact one such as INT4 (four bits per value). It lowers both the memory footprint and the bandwidth read per token, at a controlled cost in accuracy.

Time To First Token (TTFT)

The latency between submitting a prompt and receiving the first generated token. It is dominated by prompt processing and, on a cold model, by just-in-time kernel compilation.

Unified memory (UMA)

A layout in which the CPU and the iGPU share the same physical RAM rather than a dedicated VRAM pool. It enables *zero-copy*, where both processors read the same allocation without an intervening transfer, but it also makes the iGPU compete with the operating system for bandwidth.

Weight-only quantisation

A scheme in which only the weights are stored in low precision while activations stay in FP16; the dominant choice for INT4 LLMs because activations are far more sensitive to rounding than weights.

References

[1] Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. SliceGPT: Compress large language models by deleting rows and columns. *arXiv preprint arXiv:2401.15024*, 2024.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Eleiton. ollama-intel-arc: Run Ollama, Stable Diffusion and ASR with intel Arc GPU. `https://github.com/eleiton/ollama-intel-arc`, 2025–2026. Apache-2.0. Tested on Core Ultra 7 155H, Arc Graphics (Meteor Lake-P), Fedora 41.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[7] Georgi Gerganov and the llama.cpp contributors. llama.cpp. `https://github.com/ggml-org/llama.cpp`, 2023–2026.

[8] Google DeepMind. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[9] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[10] Hugging Face. Optimum Intel: Acceleration for intel hardware. `https://github.com/huggingface/optimum-intel`, 2023–2026.

[11] Intel Corporation. OpenVINO Model Server (OVMS). `https://github.com/openvinotoolkit/model_server`, 2026. Image `openvino/model_server:2026.2-gpu`.

[12] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[13] Enzo Juhel. ai-agent: an OpenAI-compatible server for Gemma 4 via optimum-intel. Field report and source tree, 2026.

[14] Enzo Juhel. Decode-throughput benchmark of INT4 LLMs on intel Iris Xe (sycl, vulkan, ovms). Primary measurements, `benchmark_results.csv`, 2026. 35 model/backend configurations, three runs each, single Iris Xe 96 EU laptop.

[15] Enzo Juhel. Intel iGPU LLM backend comparison: CPU, Vulkan, SYCL, OVMS. Field report, 2026.

[16] Enzo Juhel. Serving Gemma 4 on Iris Xe via OpenVINO graph stitching under OVMS. Field report, 2026.

[17] Roman Kazantsev. optimum-intel, branch `support_gemma_4`. `https://github.com/rkazants/optimum-intel`, 2026. Commit `eac3893`; sole source of `OVModelForVisualCausalLM` for Gemma 4 at time of writing.

[18] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2022.

[19] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[20] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

[21] Meta AI. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[22] OpenVINO Toolkit contributors. Ollama-OV: OpenVINO GenAI backend for Ollama. `https://github.com/openvinotoolkit/openvino_contrib`, 2025–2026. Module `modules/ollama_openvino`.

[23] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.